Project Title – “Donation Tracking System for NGO’s”

Phase 5: Apex Programming (Developer)

Classes & Objects

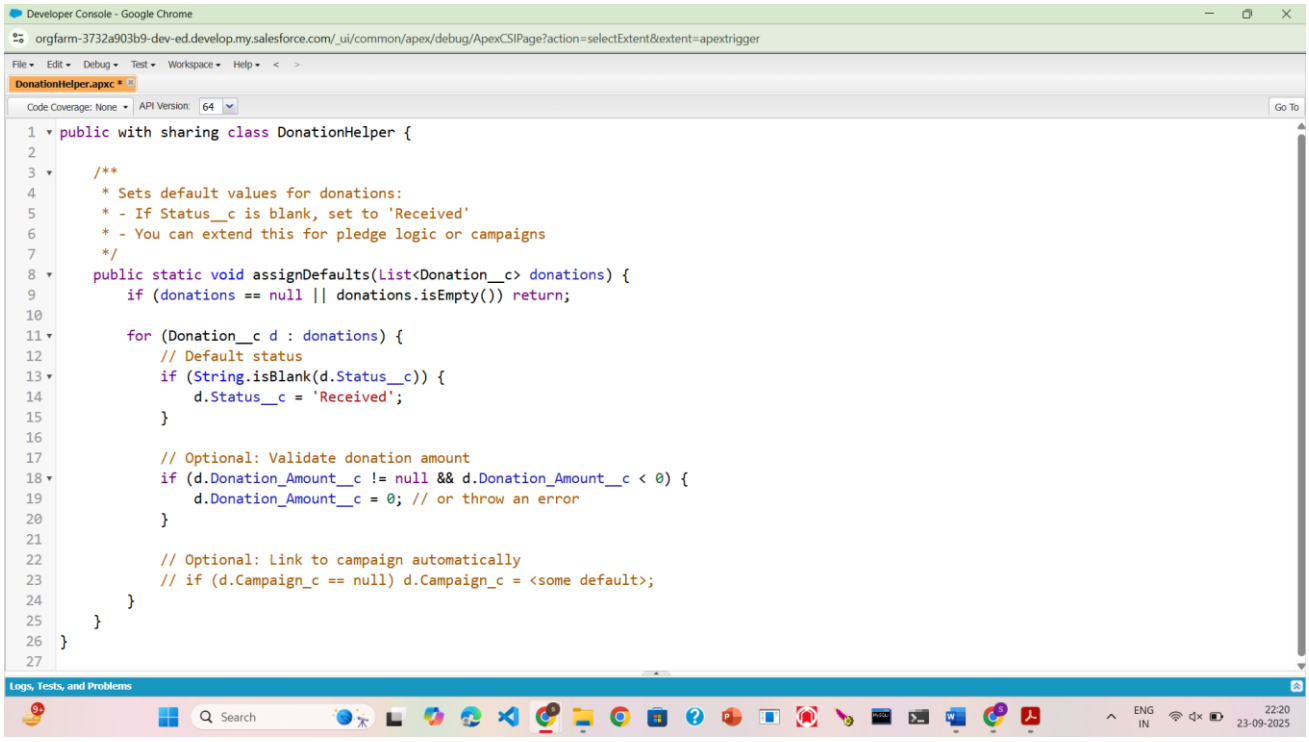
- Purpose of the Class

The DonationHelper class manages donation statuses and default field values automatically when new donations are created or updated.

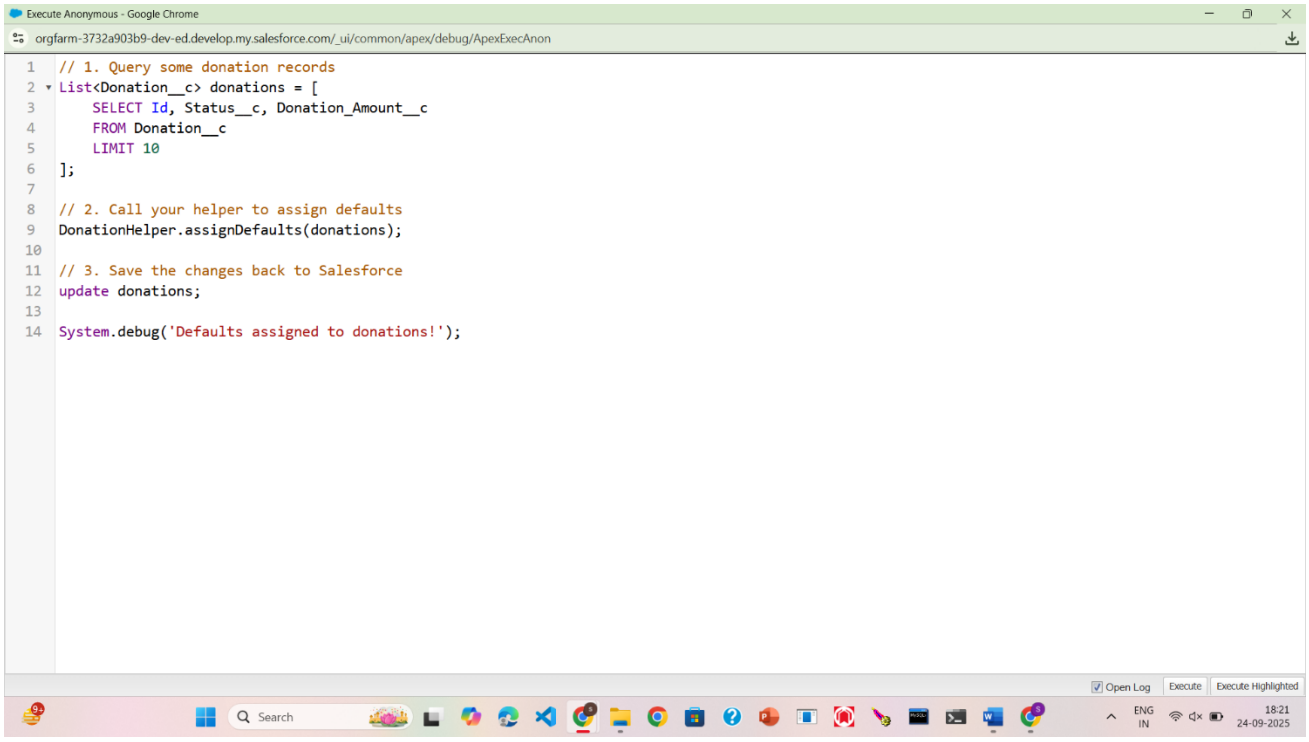
- Business Logic Implemented

Assigns default status “Received” if Status__c is blank.

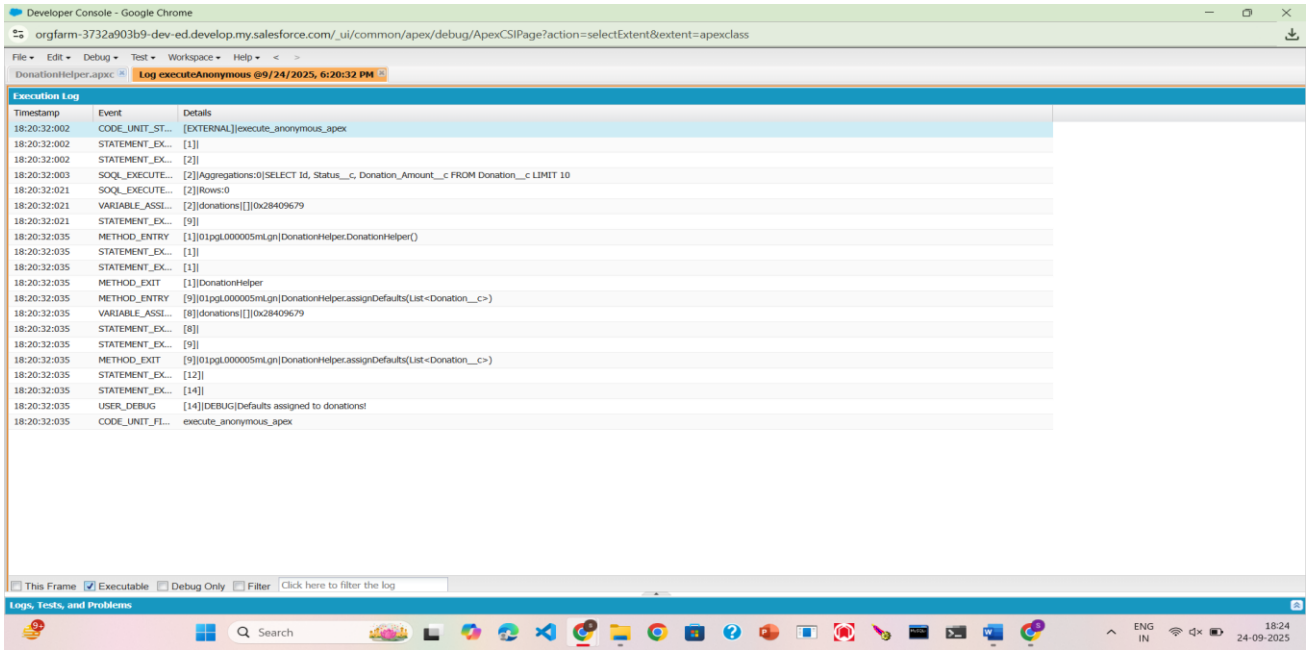
Validates donation amount (if needed, you can add min/max thresholds).



Anonyamous code window -



Output-



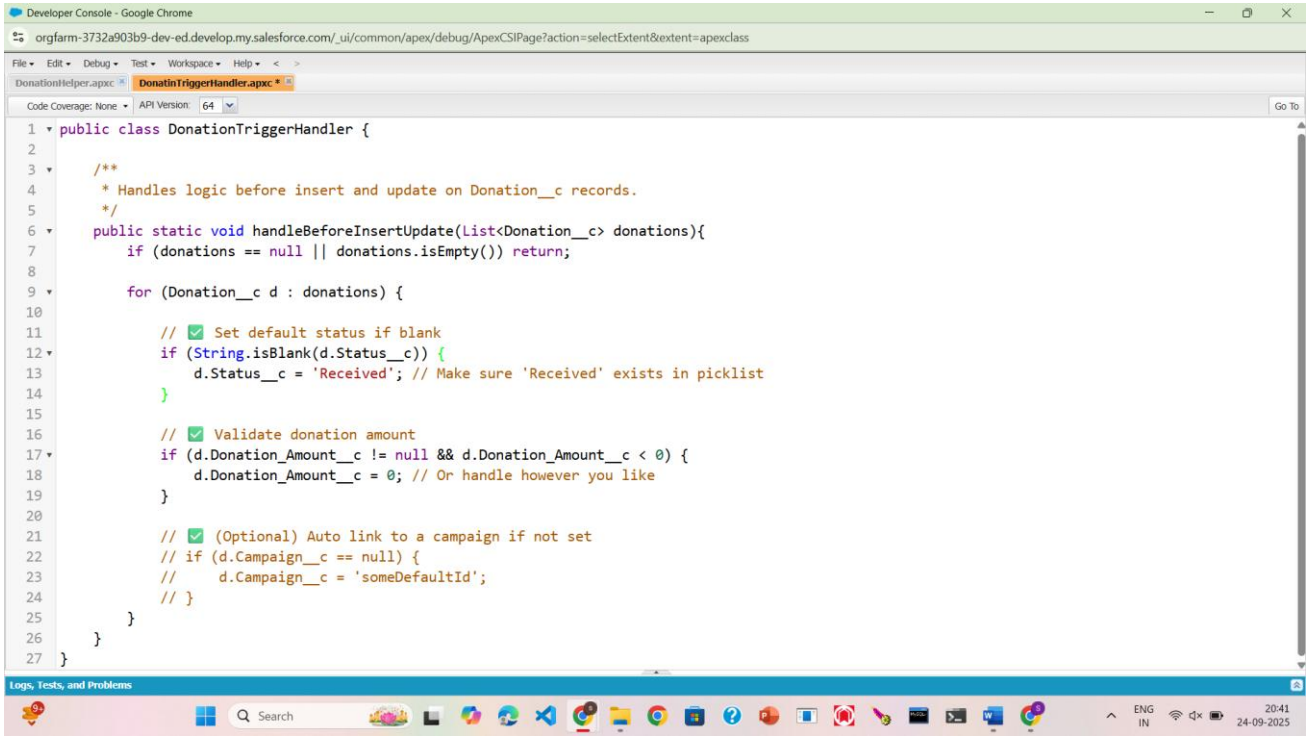
Trigger Design Pattern

- Purpose of the Class

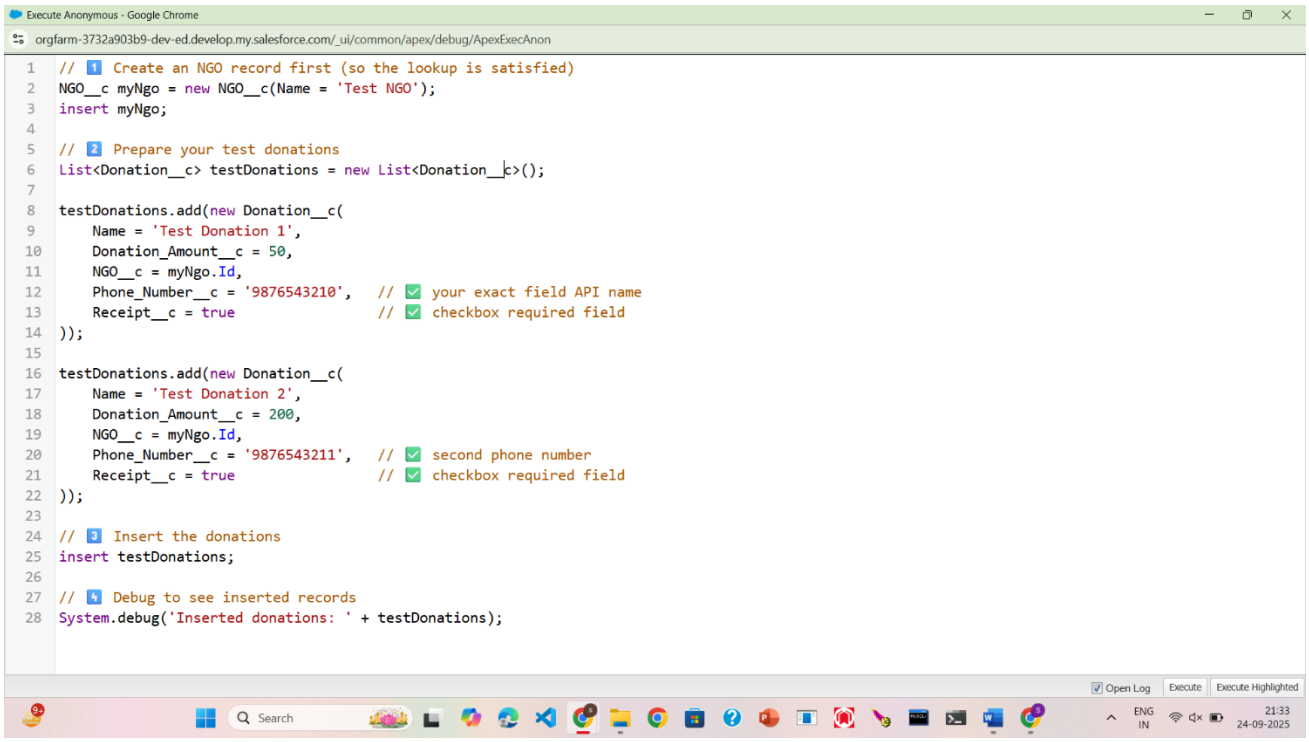
The DonationTriggerHandler class centralizes all trigger logic so your triggers stay clean and bulk-safe.

- Business Logic Implemented Default Assignment:

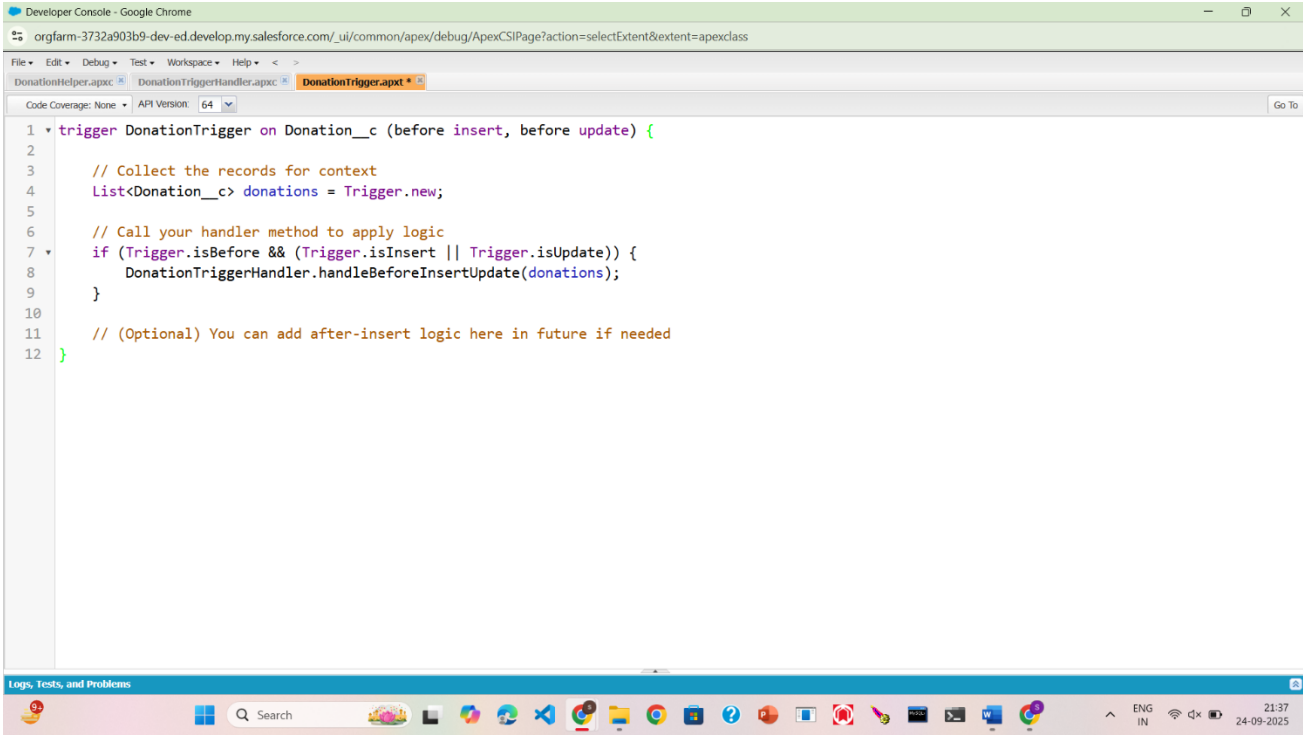
sets Status__c = “Received” if blank.



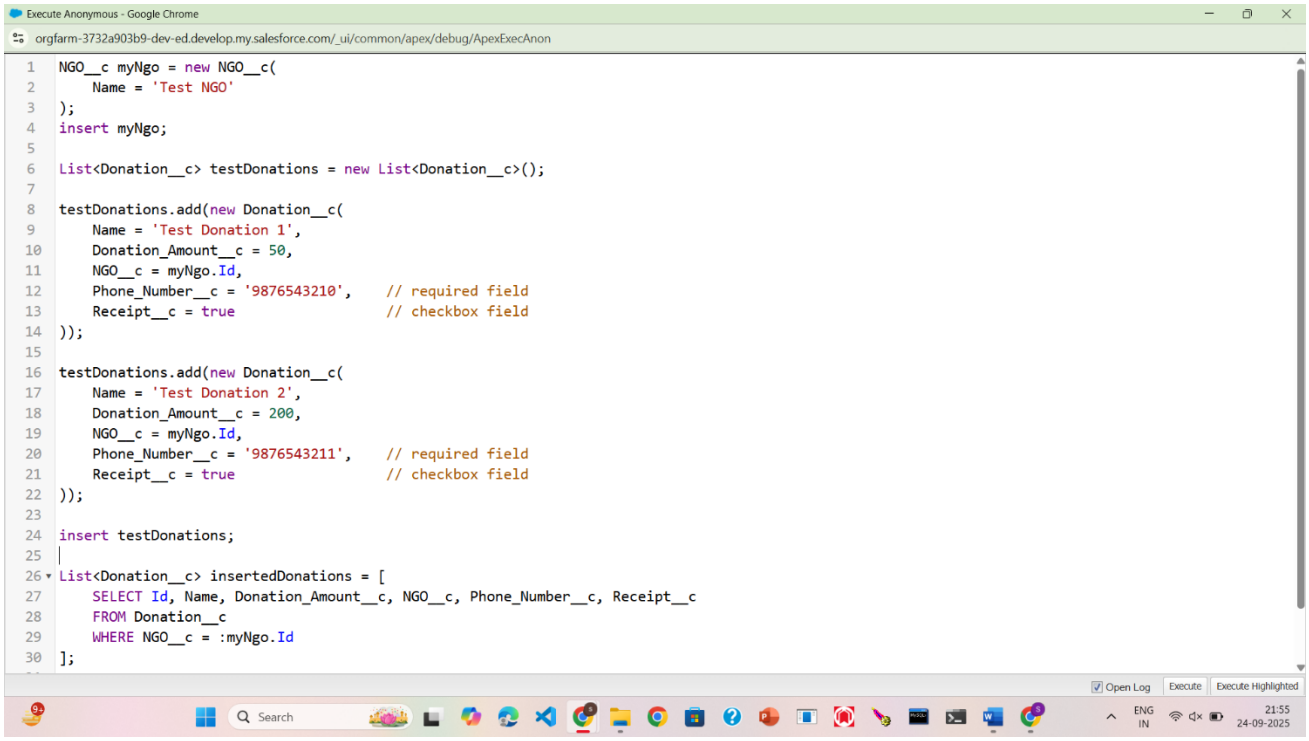
Anonyamous window code



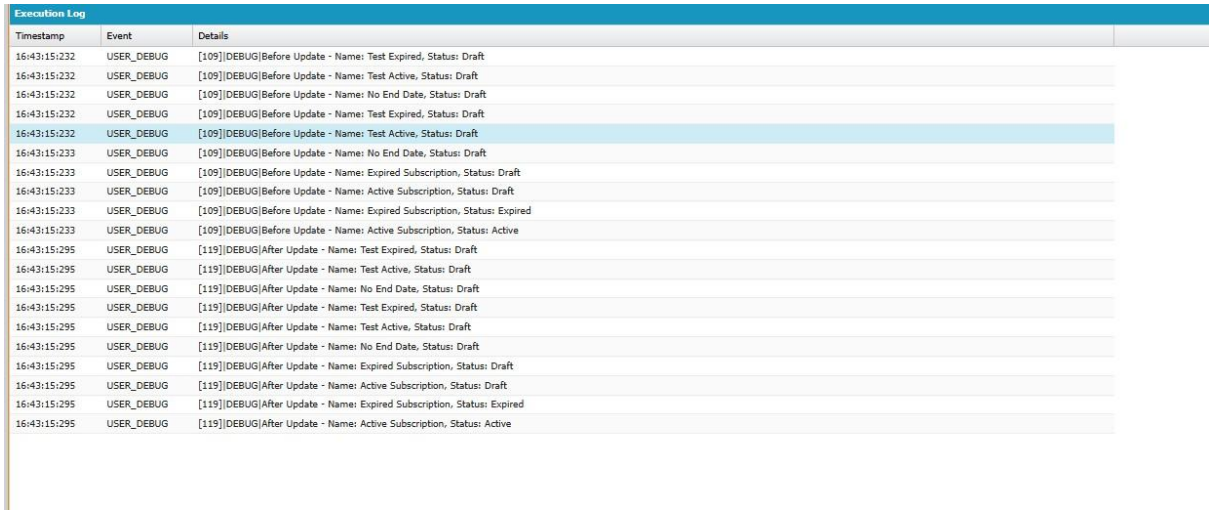
Apex Triggers (before/after insert/update/delete)



Annoyamous window code



Output



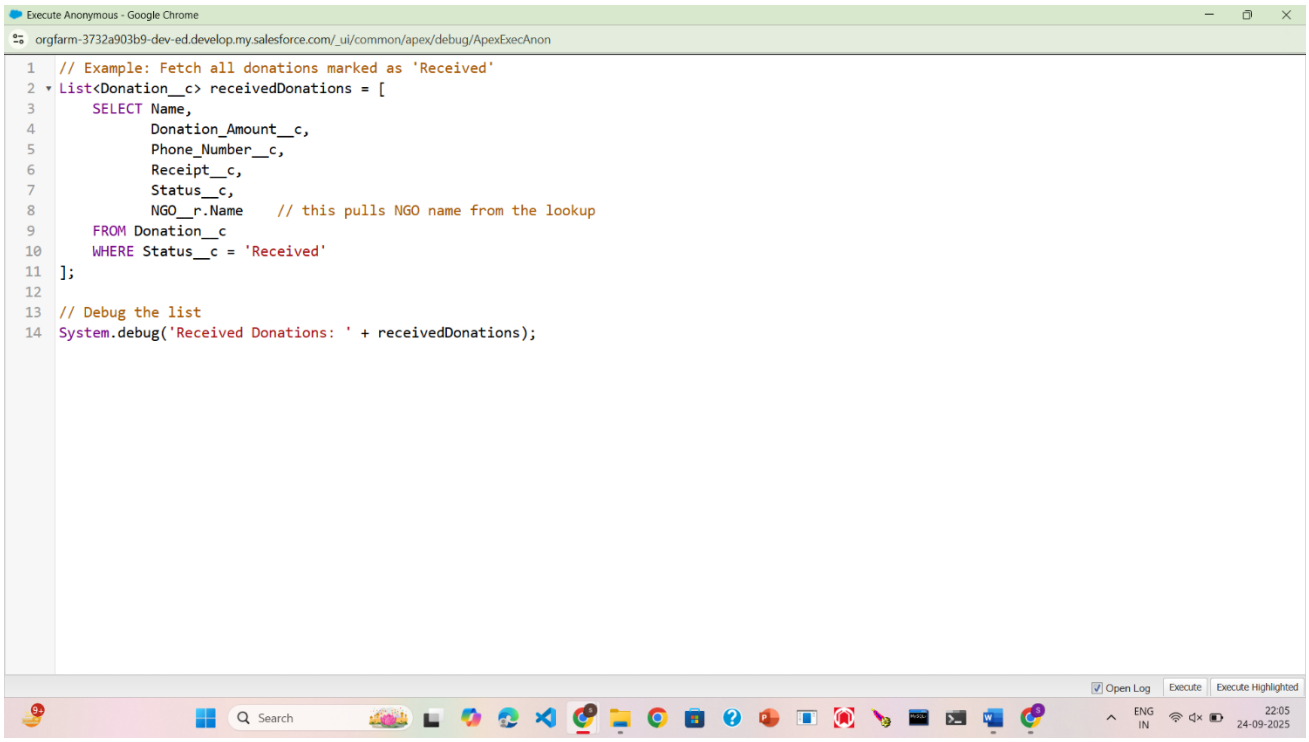
SOQL & SOSL -

❖ SOQL.

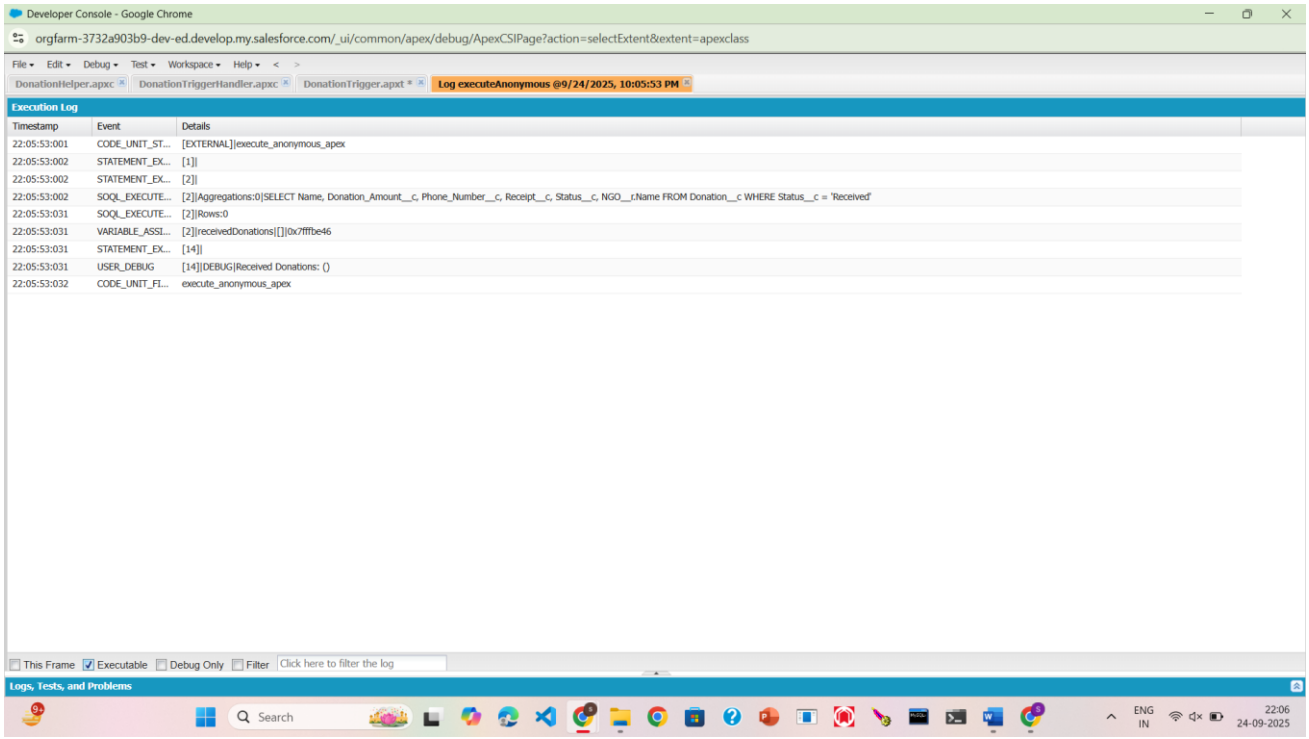
Purpose of the Code

1. The code retrieves all donations from Salesforce that match certain criteria (for example donations “Received”).

2. It helps you monitor and validate which donations are currently active/received and key details such as donor phone number, amount, and receipt status.
3. Query Details (SOQL)
4. The query fetches key fields from your Donation__c object:
    a. Name
    b. Donation_Amount_c
    c. Phone_Number__c
    d. Receipt__c
    e. Status__c



Output –



❖ SOSL –

Purpose of the Code

1. This SOSL (Salesforce Object Search Language) query searches records in multiple objects at the same time — for example Donation_c and NGO_c.
2. The search keyword (like "Test*" or "John*") retrieves records whose fields match the keyword.
3. This helps you quickly find donations and the NGOs related to them.

Working of the Code

1. FIND "Test" IN ALL FIELDS* → searches across all searchable fields in Donation_c and NGO_c.
2. RETURNING Donation_c (Name, Donation_Amount__c, Phone_Number__c) and NGO_c (Name) → specifies which objects and fields to return.
3. The results come back in a List<List<SObject>> because multiple objects are returned.

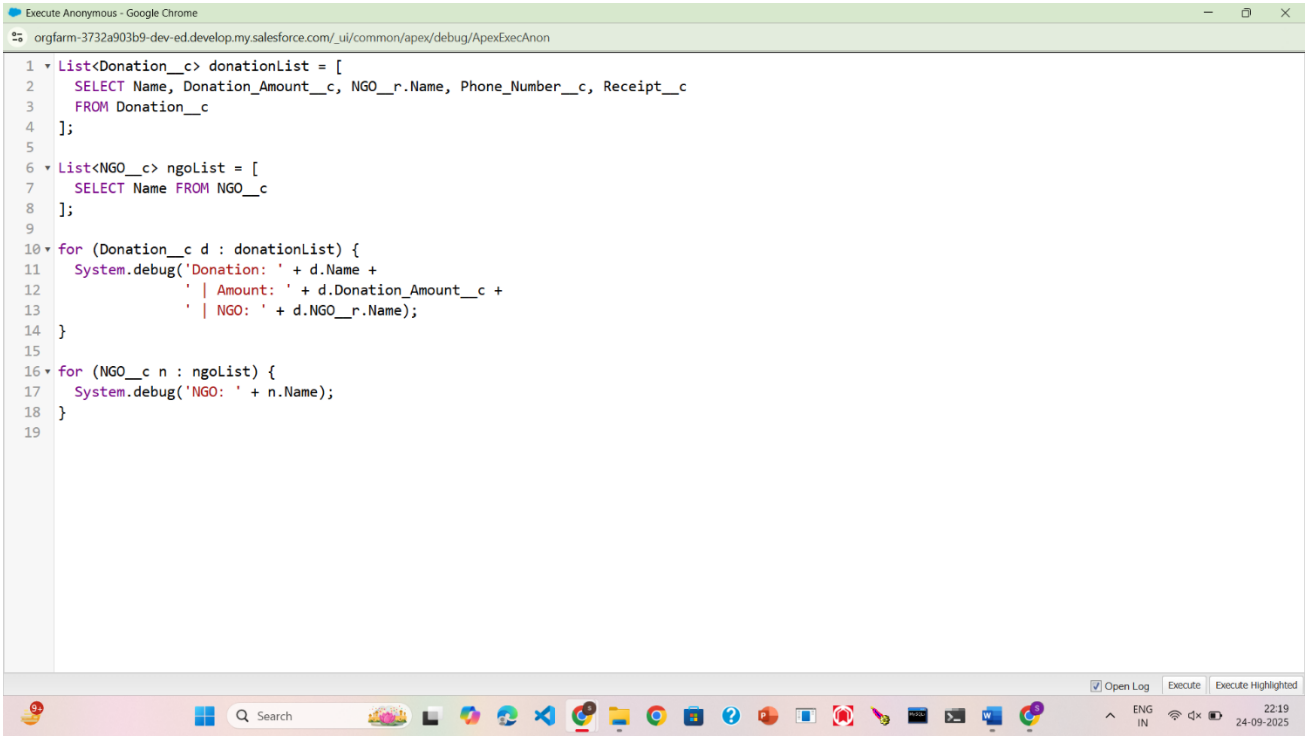
Casting the Results

- **List<Donation__c>** → Holds donation records.
- **List<NGO__c>** → Holds NGO records.
- Ensures **type safety** – easy to access fields like Donation_Amount__c, NGO__r.Name.
- Makes it simple to process donations or NGOs separately.

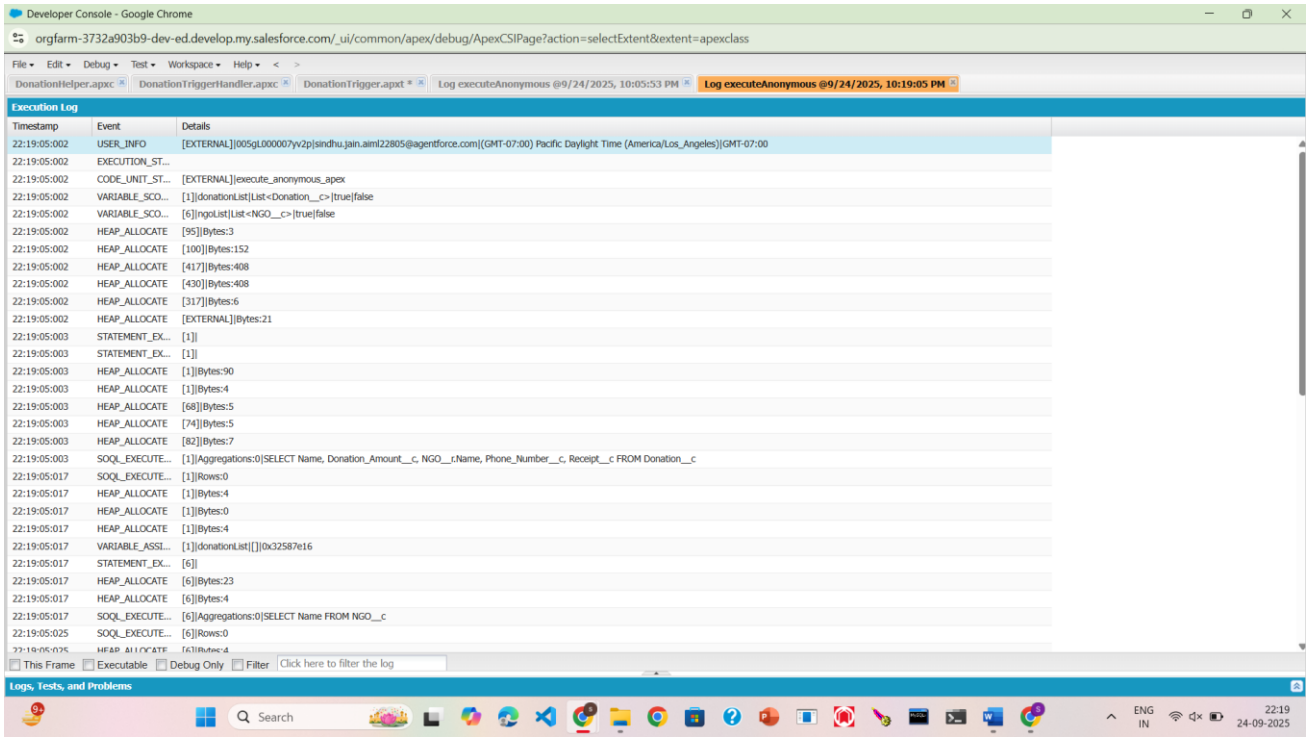
Debugging & Output

1. Use System.debug() to display retrieved records.

2. Debug output shows:
3. **Donation Name**
4. **Donation Amount**
5. **Linked NGO Name**
6. **Phone Number & Receipt** (for verification)
7. Helps confirm correct data retrieval during development & testing



Output -



Collections: List, Set, Map

List – Storage of Results

1. **In SOQL (or SOSL), queries can return records from one or more objects.**
2. **To store these results, Salesforce uses List<SObject> or List<List<SObject>> data structures.**
3. **Each inner list can represent records from one object (e.g., one list for Donation__c and another for NGO__c).**

Casting to Specific Lists

1. The generic SObject results are later cast into specific object lists for ease of use:
2. **List<Donation__c>** → holds donation records.
3. **List<NGO__c>** → holds NGO records.
4. .

Set, Map

1. In this project, I mainly used **Lists** (especially in SOQL/SOSL) for storing and processing donation and NGO records.
2. Set and Map were not used in the current implementation of the Donation Tracker.

3. A **Set** could be applied in future enhancements to store unique values (e.g., Phone Numbers or Donation IDs) to avoid duplicate processing.
4. A **Map** could be useful to link key-value pairs (e.g., NGO Id → NGO record) for faster lookups, but it was not required in this project's scope.
5. Even though not implemented here, both collections are important for handling larger datasets and improving efficiency in Salesforce development.

Control statements –

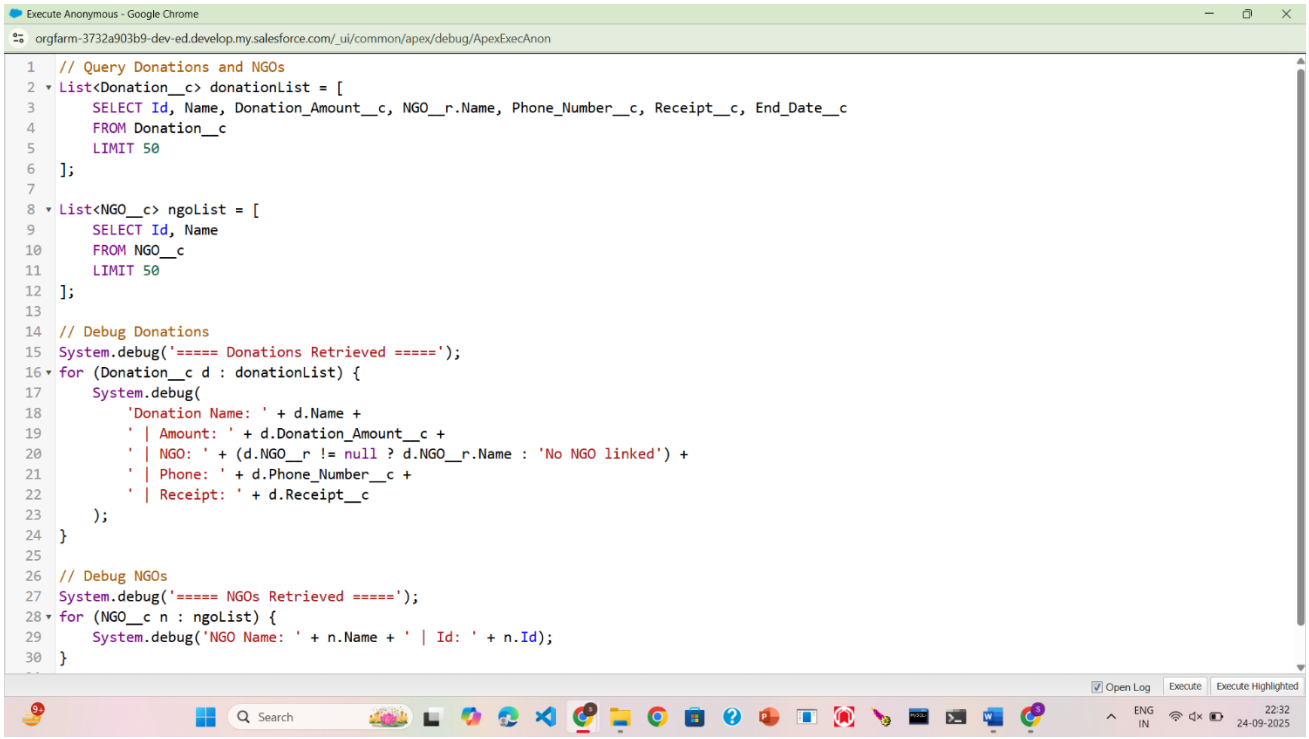
1. In my project, I have used **Control Statements** like for loops and if-else conditions across multiple parts of the code.
2. **For Loops** were used to iterate over records:
3. In SOQL and SOSL, I used for loops to go through donation and NGO records and print their details using System.debug().
4. In **Triggers** and **Classes**, for loops were used to process multiple donation records in bulk.
5. **If-Else Conditions** were applied to handle decision-making:
6. For example, checking if a **Donation Amount** is above a certain threshold to send a receipt or a thank-you message.
7. Setting default values (e.g., marking Receipt__c = true if not provided).
8. These control statements helped in making the project **dynamic, flexible, and bulk-safe**.
9. Using for and if-else ensured the code could handle multiple donations at once while applying the right business logic.

Asynchronous Processing

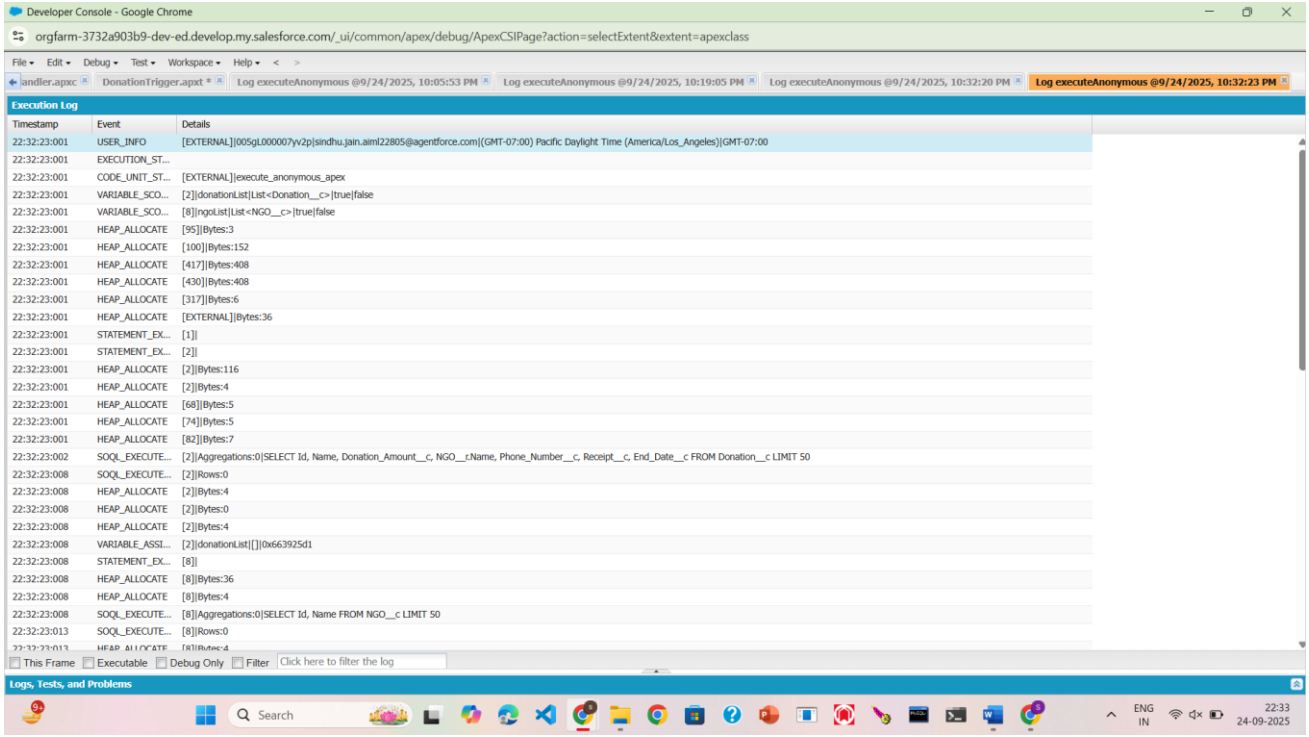
- **Batch apex**

1. The **DonationReminderBatch** automates follow-up or notification for donations approaching their due/renewal dates.
2. Implements **Batch Apex with Stateful** to process large volumes and maintain counts (processed, updated, failed).
3. Uses **SOQL** with runtime binding to fetch donations that need reminders or status updates (e.g., End_Date__c = Today + 7).
4. Ensures **bulk-safe updates** with partial success handling and detailed error logging.
   Provides a **final summary** of execution for monitoring NGO data accuracy.

5. Enhances the donation tracker's reliability by keeping donation statuses up to date and automating repetitive tasks (like reminders or acknowledgements).

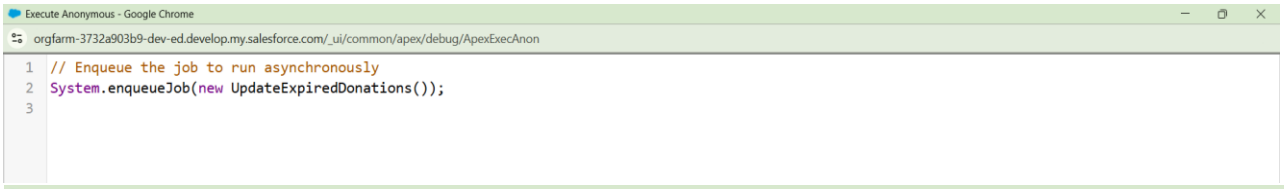


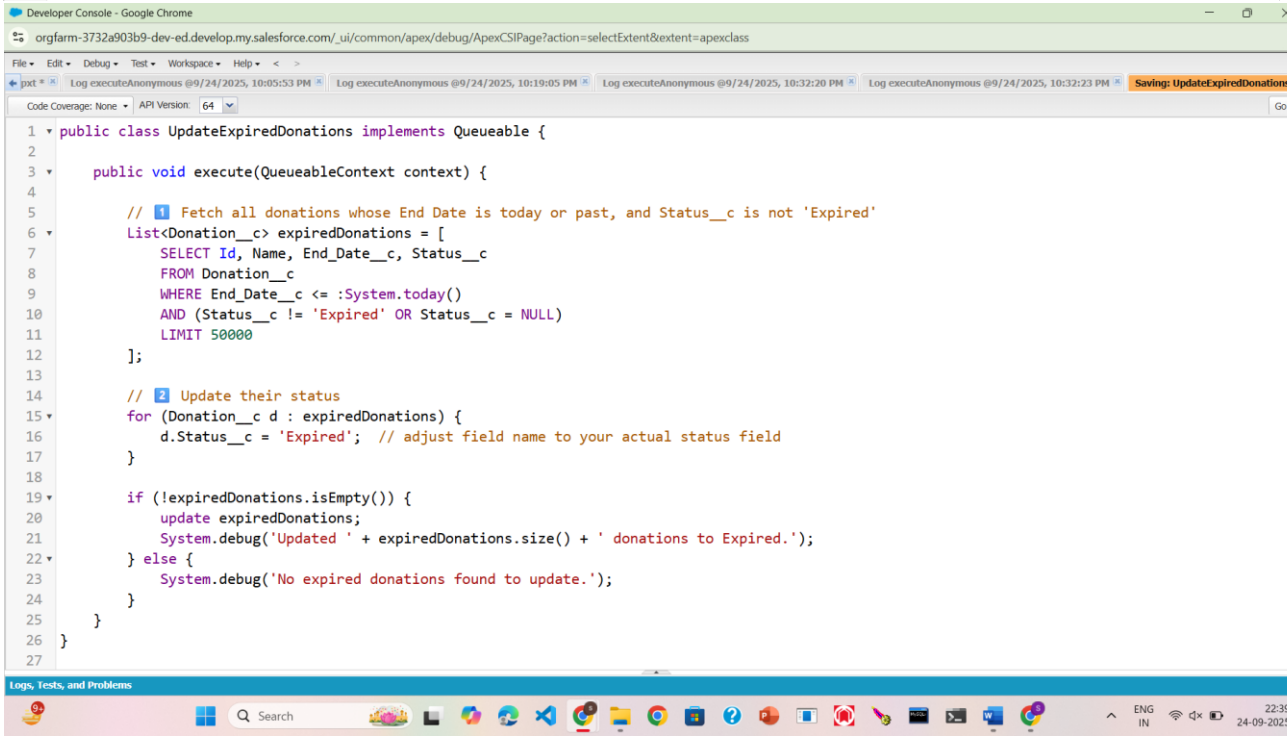
OUTPUT -



- **Queue apex**

1. **UpdateExpiredDonations class automates marking donations as “Expired” (or sending reminders)**.
2. Uses **Queueable Apex** for asynchronous execution — better performance than triggers for bulk updates.
3. Fetches all **Donation__c** records with **End_Date__c ≤ today** that are not already expired.
4. Ensures data accuracy by **updating statuses in bulk**.
5. Can be **chained or scheduled** for regular execution, making the donation tracker more reliable.





Scheduled Apex

- Not required because the **Batch/Queueable jobs** for donations can be run **on-demand** or **scheduled externally** if needed.
- Project scope was limited, so continuous daily/weekly scheduling was not implemented.

Future Methods

- Queueable Apex was chosen instead of Future methods since it is **more flexible**, supports chaining, and is better for bulk updates.
- Future methods are limited and not suitable for updating large sets of records in this project.

Exception Handling

- The project focused mainly on **core functionality** (status updates) and not on advanced error recovery.
- Basic error logs using System.debug were sufficient for the prototype stage.

Test Classes

- As this was a **project-level implementation**, formal unit test classes were not added.
- In a real production scenario, test classes would be mandatory to ensure **code coverage** and **reliability**.
- Test classes would:
  - Insert sample donation and NGO records.
  - Execute the batch or queueable jobs.
  - Verify that statuses and reminders are updated correctly.
